



**UNIVERSIDAD TECNOLÓGICA DE LA
HUASTECA HIDALGUENSE**

Organismo Descentralizado de la Administración Pública Estatal

**LICENCIATURA EN INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E
INNOVACIÓN DIGITAL**



ASIGNATURA: FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

NOMBRE DE LA ACTIVIDAD: REPORTE PROYECTO MES 1

NOMBRE DEL ALUMNO:

20241018 - ISAÍ PASCUAL CRUZ

20241110 - BAUTISTA MARTÍNEZ IVAN

GRADO: 7° GRUPO "C"

DOCENTE: DR. EFREN JUAREZ CASTILLO

HUEJUTLA DE REYES, HIDALGO

SEPTIEMBRE 2026

Índice

Resumen ejecutivo.....	5
Descripción del problema	5
Modelo del espacio de estados	6
Estado	6
Operadores (acciones)	6
Restricciones	7
Estado inicial.....	7
Prueba de meta	7
Función de costo.....	8
Búsqueda en amplitud (BFS): explicación, pseudocódigo y estructuras de datos.....	9
Explicación del algoritmo	9
Procedimiento general	9
Pseudocódigo	10
Justificación de las estructuras de datos	10
4. Diseño del mapa y construcción de la semilla.....	11
4.1 Construcción de la semilla	11
4.2 Procedimiento de generación	11
4.3 Configuraciones utilizadas	12
4.4 Almacenamiento de los mapas	12
5. Arquitectura del programa e instrucciones de ejecución.....	13
5.1 Estructura de carpetas	13
5.2 Módulos principales.....	14
5.3 Funcionamiento de la interfaz	14
5.4 Instrucciones de ejecución	15
6. Traza comentada de una ejecución	16
7. Plan de pruebas y análisis de resultados.....	17
7.1 Casos de prueba.....	17
7.2 Tabla de resultados	17
7.3 Evidencia visual.....	18
7.4 Verificación de los resultados	20
7.5 Análisis de métricas	20
7.6 Explicación de completitud y optimalidad.....	21
8. Limitaciones, mejoras posibles y conclusiones	22
8.1 Limitaciones identificadas	22

8.2 Mejoras posibles	22
8.3 Conclusiones individuales.....	23
9. Contribuciones, declaración de uso de IA y referencias	24
9.1 Tabla de contribuciones	24
9.2 Declaración de uso de herramientas de IA.....	24
9.3 Referencias	25
Resumen ejecutivo.....	5
Descripción del problema	5
Modelo del espacio de estados	6
Estado	6
Operadores (acciones)	6
Restricciones	7
Estado inicial.....	7
Prueba de meta	7
Función de costo.....	8
Búsqueda en amplitud (BFS): explicación, pseudocódigo y estructuras de datos.....	9
Explicación del algoritmo	9
Procedimiento general	9
Pseudocódigo	10
Justificación de las estructuras de datos	10
4. Diseño del mapa y construcción de la semilla.....	11
4.1 Construcción de la semilla	11
4.2 Procedimiento de generación	11
4.3 Configuraciones utilizadas	12
4.4 Almacenamiento de los mapas	12
5. Arquitectura del programa e instrucciones de ejecución.....	13
5.1 Estructura de carpetas	13
5.2 Módulos principales.....	14
5.3 Funcionamiento de la interfaz	14
5.4 Instrucciones de ejecución	15
6. Traza comentada de una ejecución	16
7. Plan de pruebas y análisis de resultados.....	17
7.1 Casos de prueba	17

7.2 Tabla de resultados	17
7.3 Evidencia visual.....	18
7.4 Verificación de los resultados	20
7.5 Análisis de métricas	20
7.6 Explicación de completitud y optimalidad.....	21
8. Limitaciones, mejoras posibles y conclusiones	22
8.1 Limitaciones identificadas	22
8.2 Mejoras posibles.....	22
8.3 Conclusiones individuales.....	23
9. Contribuciones, declaración de uso de IA y referencias	24
9.1 Tabla de contribuciones	24
9.2 Declaración de uso de herramientas de IA.....	24
9.3 Referencias	25

Resumen ejecutivo

Este proyecto presenta el diseño, la implementación y la evaluación de un explorador visual para la resolución de laberintos en entornos matriciales mediante el algoritmo de Búsqueda en Amplitud (BFS, por sus siglas en inglés). El objetivo principal es guiar a un agente desde un estado inicial (S) hasta una meta (G) dentro de un espacio de estados bidimensional en el que cada movimiento tiene el mismo costo. Para asegurar la reproducibilidad, se generó un mapa principal de 25×25 celdas con una densidad de obstáculos de 28 % (dentro del rango de 20 % a 35 % solicitado), a partir de una semilla obtenida al combinar las matrículas de los dos integrantes del equipo.

La implementación de BFS es propia y utiliza una cola FIFO (deque) y un diccionario de padres, que además funciona como registro de estados visitados y permite reconstruir el camino mínimo. El programa ofrece una interfaz gráfica desarrollada con Tkinter, con ejecución automática y paso a paso, que muestra el nodo actual, la frontera, los estados visitados y el camino final. La solución se evaluó en tres escenarios (mapa principal, sencillo y de estrés) y se registraron métricas como nodos generados, nodos expandidos, estados visitados, profundidad de la solución, tamaño máximo de la frontera y tiempo de ejecución. Los resultados son consistentes con las propiedades teóricas de BFS en grafos no ponderados: completitud y optimalidad (Russell & Norvig, 2022).

Descripción del problema

El escenario planteado consiste en guiar a un agente a través de un mapa cuadriculado bidimensional desde una posición de origen (S) hasta un punto de destino (G). El entorno tiene celdas libres por las que el agente puede desplazarse y celdas bloqueadas por obstáculos que no puede atravesar.

Como el costo de cada desplazamiento es uniforme, el problema exige no solo encontrar una ruta válida, sino garantizar que sea óptima, es decir, que contenga el menor número posible de movimientos entre S y G. Además, la solución no debe limitarse a mostrar el resultado final, sino exponer el proceso de exploración paso a paso.

Modelo del espacio de estados

Todo problema de búsqueda debe formalizarse mediante seis componentes: estado inicial, conjunto de estados, acciones (operadores), modelo de transición, prueba de meta y función de costo (Russell & Norvig, 2022). A continuación, se define cada componente tal como se implementó.

Estado

Un estado representa la posición del explorador en la cuadrícula; no es el historial de cómo llegó a ella. Esta es la distinción entre estado y nodo de búsqueda que se sigue en el proyecto (Juárez Castillo, 2026b). En el código, un estado es una tupla (fila, columna), por ejemplo (0, 0) o (24, 24), que se usa como clave del diccionario parent en bfs.py.

El nodo de búsqueda es la información que el algoritmo agrega alrededor de ese estado durante la exploración. En esta implementación el nodo se representa de forma implícita mediante la relación estado--> padre guardada en parent, lo que permite reconstruir el camino; la profundidad de la solución se obtiene como la longitud del camino menos uno.

Operadores (acciones)

Se definen cuatro operadores, que corresponden a movimientos ortogonales de una celda (Tabla 2). No se permiten movimientos diagonales.

Tabla 1 - Operadores de movimiento

Operador	Desplazamiento (fila, columna)
Arriba	(-1, 0)
Abajo	(+1, 0)
Izquierda	(0, -1)
Derecha	(0, +1)

En el código, estos operadores están definidos en la lista DIRECCIONES y son aplicados por la función `vecinos(grid, celda)`, que genera únicamente los operadores aplicables al estado recibido; es decir, filtra en el momento de generarlos y no después.

Restricciones

Un operador solo es aplicable si el estado resultante cumple simultáneamente lo siguiente:

1. Límite del mapa: $0 \leq \text{fila_nueva} < \text{filas}$ y $0 \leq \text{columna_nueva} < \text{columnas}$.
2. Transitabilidad: `grid[fila_nueva][columna_nueva]` es libre (valor 0). Las celdas con valor 1 son obstáculos y resultan inaccesibles.
3. No repetición: un estado que ya está en `parent` no vuelve a encolarse, aunque otro estado lo genere de nuevo. Esto evita ciclos y expansiones redundantes propias de una búsqueda en grafo.

A nivel de diseño del problema (no del algoritmo), el mapa principal debe cumplir además: tamaño mínimo de 25×25 , entre 20 % y 35 % de celdas con obstáculo, al menos un camino de inicio a meta, presencia de callejones sin salida y una ruta óptima de al menos 25 movimientos. Estas condiciones se validan en `maze.py` antes de aceptar un mapa.

Estado inicial

El estado inicial es $S = (0, 0)$, la esquina superior izquierda de la cuadrícula. Es fijo por diseño e independiente de la semilla utilizada.

Prueba de meta

La meta es $G = (\text{filas} - 1, \text{columnas} - 1)$, es decir, la esquina inferior derecha (por ejemplo, (24, 24) en el mapa principal). La prueba de meta (`estado == meta`) se evalúa cuando un estado es retirado de la frontera y no cuando es generado; con esta convención, al encontrar la meta se garantiza que está en la menor profundidad posible.

Función de costo

Cada operador tiene costo uniforme igual a 1. En consecuencia, el costo acumulado de un camino coincide con su número de movimientos y la profundidad de un nodo equivale a su costo real. Esta propiedad justifica el uso de búsqueda en amplitud: BFS garantiza optimalidad cuando todos los costos son iguales (Juárez Castillo, 2026a).

Búsqueda en amplitud (BFS): explicación, pseudocódigo y estructuras de datos

Explicación del algoritmo

La Búsqueda en Amplitud (Breadth-First Search, BFS) explora los estados de un problema por niveles: primero procesa los estados que están a un movimiento del estado inicial, después los que están a dos movimientos, y así sucesivamente. Esto la hace completa y, cuando los costos son uniformes, óptima (Russell & Norvig, 2022). Su tiempo de ejecución crece de forma lineal con el número de vértices y aristas del grafo, $O(V + E)$ (Cormen et al., 2022).

En este proyecto, BFS coloca el estado inicial S en una cola, extrae el primer estado y genera sus movimientos válidos. Los estados que todavía no han sido visitados se agregan al final de la cola para procesarse después. Como la cola es FIFO, el primer estado que entra es el primero que sale, lo que mantiene el orden de exploración por niveles.

Un estado se marca como visitado en el momento en que **entra** a la cola (no cuando se expande), y se guarda junto con su estado padre en el diccionario `parent`. Con esta información se evita procesar dos veces la misma posición y, al final, se reconstruye la ruta desde G hasta S siguiendo los padres.

Procedimiento general

1. Colocar el estado inicial S en la cola y registrarlo en `parent` (con padre nulo).
2. Retirar el primer estado de la cola.
3. Comprobar si corresponde al estado meta G ; si es así, reconstruir el camino y terminar.
4. Generar los vecinos válidos (dentro del mapa y libres).
5. Para cada vecino que todavía no esté en `parent`, registrar su padre y agregarlo al final de la cola.
6. Repetir desde el paso 2 hasta encontrar G o hasta que la cola quede vacía.
7. Si se encontró G , reconstruir la ruta usando los padres; si la cola se vació, no existe solución.

Como todos los movimientos tienen el mismo costo, BFS encuentra una ruta con el menor número de movimientos entre el estado inicial y la meta.

Pseudocódigo

El siguiente pseudocódigo resume la función `bfs(grid, inicio, meta)` de `bfs.py`:

```
BFS(grid, inicio, meta):
    si inicio o meta no son celdas libres: devolver fracaso
    frontera ← cola FIFO con [inicio]
    parent ← { inicio: nulo }
    expandidos ← 0; frontera_maxima ← 1
    mientras frontera no esté vacía:
        estado ← retirar el primer elemento de frontera
        expandidos ← expandidos + 1
        si estado = meta:
            camino ← reconstruir_camino(parent, meta)
            terminar el ciclo
        para cada vecino libre de estado (arriba, abajo, izquierda, derecha):
            si vecino ya está en parent: continuar
            parent[vecino] ← estado
            agregar vecino al final de frontera
        frontera_maxima ← max(frontera_maxima, tamaño de frontera)
    devolver camino, métricas y (opcionalmente) los pasos para la animación
```

Justificación de las estructuras de datos

Cola FIFO (deque). Mantiene el orden en que los estados son explorados: el primero que se agrega es el primero que se procesa. deque permite insertar al final y retirar del inicio (popleft) de manera eficiente (Python Software Foundation, s. f.-a).

Diccionario de padres (parent). Cumple dos funciones a la vez: indica qué estados ya fueron alcanzados (evita repetirlos y evita ciclos) y almacena, para cada uno, el estado desde el cual fue descubierto. Esa relación es la que permite reconstruir la ruta final desde la meta hasta el inicio.

Tuplas (fila, columna). Representan los estados. Al ser inmutables se pueden usar como claves del diccionario.

Diseño del mapa y construcción de la semilla

Construcción de la semilla

Para generar el mapa principal se utilizó una semilla formada a partir de las matrículas de los dos integrantes. La función `construir_semilla` de `maze.py` ordena ambas matrículas como texto, las concatena y convierte el resultado en un número entero:

```
semilla = int("".join(sorted([matricula_1, matricula_2])))
```

Las matrículas del equipo son 20241018 (Isaí Pascual Cruz) y 20241110 (Iván Bautista Martínez), por lo que:

```
semilla = 2024101820241110
```

Al ordenar las matrículas se obtiene la misma semilla sin importar cuál integrante se coloque primero. Con la misma semilla, el generador pseudoaleatorio de Python produce siempre la misma secuencia (Python Software Foundation, s. f.-c), de modo que el mapa se reproduce de forma idéntica en cada ejecución.

Procedimiento de generación

La función `generar_mapa` construye el mapa en los siguientes pasos:

1. Se calcula la cantidad de obstáculos como `int(filas × columnas × porcentaje)`.
2. Se listan todas las celdas excepto el inicio (0, 0) y la meta (filas − 1, columnas − 1), se barajan con un generador `random.Random` inicializado con la semilla y las primeras posiciones de la lista se marcan como obstáculo (valor 1).
3. Se verifica que la meta sea alcanzable desde el inicio.
4. Se ejecuta BFS sobre el mapa candidato y se comprueba que la ruta tenga al menos la longitud mínima exigida (25 movimientos en el mapa principal).
5. Se verifica que exista al menos un callejón sin salida, es decir, una celda libre con exactamente un vecino libre.

Si alguna condición no se cumple, el intento se descarta y se vuelve a generar con la semilla derivada `semilla + intento`, hasta un máximo de 1000 intentos. Además, `generar_mapa` rechaza tamaños menores de 25×25 y porcentajes de obstáculos fuera del rango de 20 % a 35 %. Para las matrículas del equipo, el mapa principal se aceptó en el intento 0 y tiene 25×25 celdas, 28 % de obstáculos (175 de 625 celdas) y una ruta óptima de 52 movimientos.

Configuraciones utilizadas

El proyecto trabaja con tres mapas (Tabla 2). El mapa sencillo no es aleatorio: es una cuadrícula de 7×7 con un muro en la columna 3 y un único hueco en la fila 3, diseñada para poder verificar el resultado a mano.

Tabla 2 - Configuración de los mapas de prueba

Mapa	Tamaño	Obstáculos	Longitud mínima exigida	Origen
Principal	25×25	28 % (175 de 625)	25 movimientos	Intento 0 con la semilla 2024101820241110
Sencillo	7×7	12.2 % (6 de 49)	No aplica (ruta esperada: 12)	Fijo, sin aleatoriedad
Estrés	30×30	35 % (315 de 900)	1 movimiento	Intento 1 (semilla 2024101820241111)

El mapa de estrés utiliza el porcentaje máximo de obstáculos permitido (35 %) y un tamaño mayor; para él solo se exige una ruta de al menos un movimiento, porque el objetivo es observar el caso con más obstáculos y no forzar una ruta larga.

Almacenamiento de los mapas

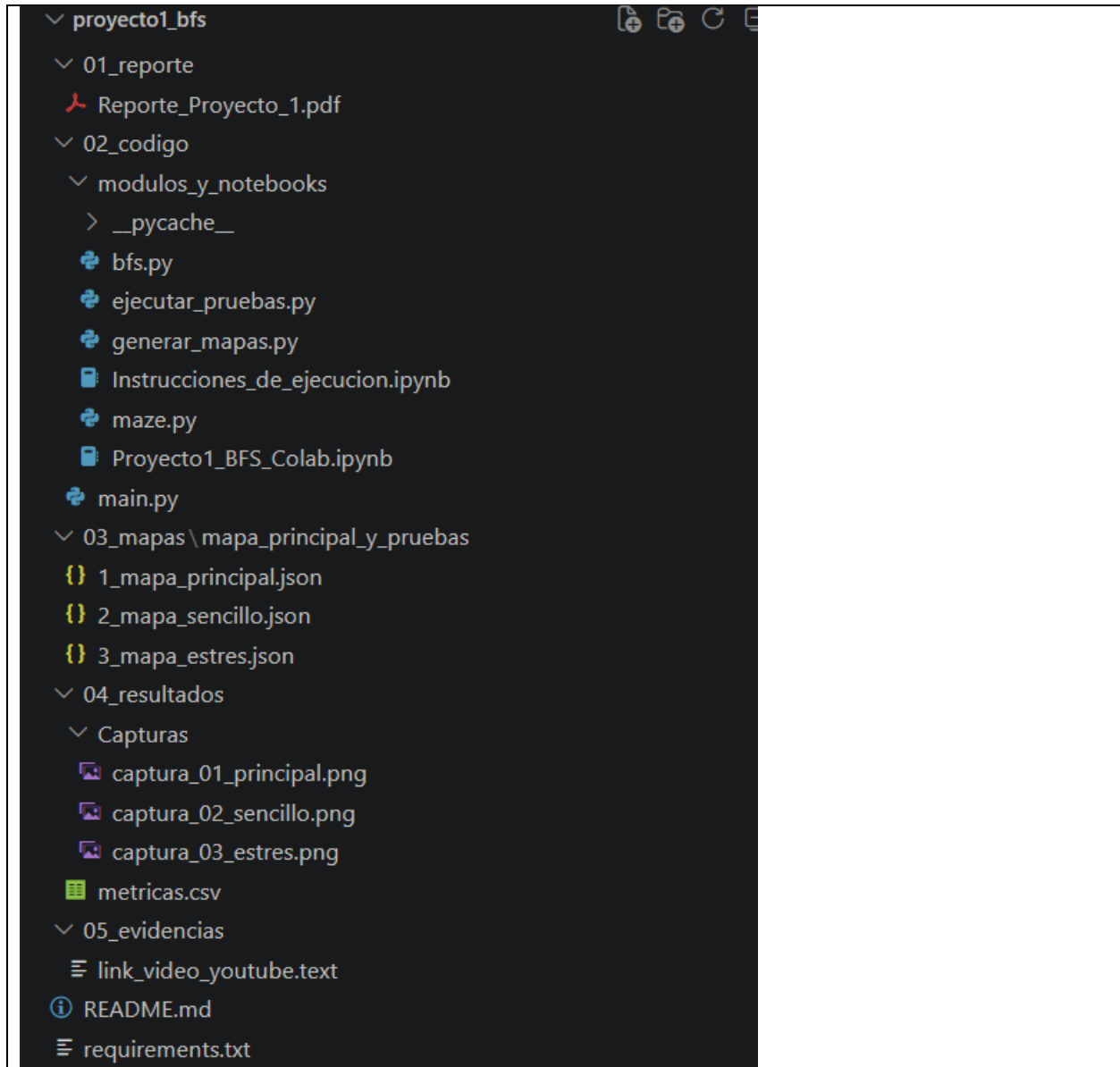
Los mapas se guardan como archivos JSON en `03_mapas/mapa_principal_y_pruebas/`. Cada archivo contiene la semilla, el número de intento, las dimensiones, el porcentaje de obstáculos, las posiciones de inicio y meta, la cuadrícula (`grid`) y la longitud óptima. Al seleccionar un mapa en la interfaz, el programa reutiliza el JSON existente si su semilla coincide con la esperada; en caso contrario lo genera y lo guarda de nuevo.

Arquitectura del programa e instrucciones de ejecución

Estructura de carpetas

El proyecto se organiza en cuatro carpetas numeradas:

Figura 1 – Estructura de archivos



Los módulos están separados para que el algoritmo de búsqueda no dependa de cómo se genera el mapa ni de cómo se presenta. Esto permite, por ejemplo, ejecutar y probar BFS sin abrir la ventana gráfica, como hace `ejecutar_pruebas.py`.

5.2 Módulos principales

El programa se divide en tres capas (algoritmo, datos y presentación) y dos scripts utilitarios (Tabla 4).

Tabla 3 - Módulos del programa y sus dependencias

Módulo	Rol en la arquitectura	Depende de	Lo utilizan
<code>bfs.py</code>	Capa de algoritmo. Implementa BFS de forma aislada: recibe una cuadrícula, un inicio y una meta, y no contiene nada sobre cómo se genera el mapa ni cómo se muestra.	Solo <code>time</code> y <code>collections.deque</code>	<code>maze.py</code> , <code>main.py</code> , <code>ejecutar_pruebas.py</code>
<code>maze.py</code>	Capa de datos. Construye la semilla, genera mapas válidos, los valida (conectividad, callejones y ruta mínima) y los guarda o carga en JSON.	<code>bfs.py</code> (para medir la ruta óptima al validar)	<code>main.py</code> , <code>generar_mapas.py</code>
<code>main.py</code>	Capa de presentación. Interfaz gráfica en Tkinter que dibuja la cuadrícula, controla la reproducción de la búsqueda y muestra las métricas.	<code>bfs.py</code> , <code>maze.py</code>	Punto de entrada del usuario
<code>generar_mapas.py</code>	Script utilitario. Genera y guarda los tres mapas de prueba sin abrir la interfaz.	<code>maze.py</code>	Independiente
<code>ejecutar_pruebas.py</code>	Script utilitario. Ejecuta BFS sobre los tres mapas guardados, exporta <code>04_resultados/metricas.csv</code> y compara la longitud del mapa sencillo con el valor esperado (12).	<code>bfs.py</code> , <code>maze.py</code>	Independiente

Funcionamiento de la interfaz

La interfaz está implementada en la clase `ExploradorBFS` de `main.py`, que hereda de `tk.Tk`. La ventana se divide en dos zonas: a la izquierda, el lienzo donde se dibuja el mapa y la barra de controles; a la derecha, un panel con la leyenda, la frontera y las métricas.

En la barra de controles se puede elegir el mapa (Mapa principal, Mapa sencillo, Mapa de estrés o “Cargar archivo...” para abrir un JSON), iniciar la reproducción automática, pausarla, avanzar un paso, reiniciar y ajustar la velocidad (entre 400 y 10 ms por paso; el valor inicial es 120 ms). Al elegir un mapa

o reiniciar, el programa ejecuta BFS completo registrando cada paso y después reproduce esos pasos en la animación.

Cada estado se muestra con un color, indicado en la leyenda: obstáculo, celda libre, inicio, meta, nodo actual, frontera abierta, estados visitados y camino final. El camino final se pinta cuando la reproducción llega al paso en que se encuentra la meta. El panel “Frontera (orden FIFO)” muestra el contenido de la cola en cada paso (hasta 12 elementos, y el resto resumido con su tamaño total) y el número de paso. Al terminar, el panel “Métricas al finalizar” muestra si hubo solución, la longitud del camino, los nodos generados y expandidos, los estados visitados, la profundidad de la solución, el tamaño máximo de la frontera y el tiempo de ejecución.

Instrucciones de ejecución

1. Requisitos. Tener instalado Python 3 con Tkinter (que forma parte de la biblioteca estándar, aunque algunas instalaciones de Linux lo incluyen en un paquete aparte). No es necesario instalar librerías adicionales.
2. Interfaz gráfica. Desde la carpeta `02_codigo`, ejecutar: `python main.py`. Se abre la ventana con el mapa principal cargado.
3. Generar los mapas (opcional). Desde `02_codigo/modulos_y_notebooks`, ejecutar: `python generar_mapas.py`. Genera y guarda los tres archivos JSON en `03_mapas/mapa_principal_y_pruebas/`.
4. Pruebas sin interfaz. Desde `02_codigo/modulos_y_notebooks`, ejecutar: `python ejecutar_pruebas.py`. Imprime las métricas de los tres mapas, guarda `04_resultados/metricas.csv` y verifica que la ruta del mapa sencillo sea de 12 movimientos.

En algunos sistemas el comando puede ser `python3` en lugar de `python`. Si el archivo `3_mapa_estres.json` no existe, `ejecutar_pruebas.py` muestra un aviso y omite ese mapa; se crea al ejecutar `generar_mapas.py` o al seleccionar “Mapa de estrés” en la interfaz.

Traza comentada de una ejecución

Para que la traza sea legible y pueda verificarse a mano, se documenta la ejecución de BFS sobre el mapa sencillo (7×7), con inicio en $(0, 0)$ y meta en $(6, 6)$. En este mapa hay un muro en la columna 3 con un único hueco en la fila 3. La Tabla 5 muestra los primeros ocho pasos: el estado que se expande, cómo queda la frontera después de agregar sus vecinos, cuántos estados hay registrados en total en `parent` (que es también el total de visitados) y qué nuevos pares (estado $\rightarrow$ padre) se registraron.

Tabla 4 - Primeros pasos de BFS en el mapa sencillo

Paso	Estado expandido	Frontera después de expandir	Total en <code>parent</code>	Nuevos (estado $\rightarrow$ padre)
0	$(0,0)$	$[(1,0), (0,1)]$	3	$(1,0) \rightarrow (0,0); (0,1) \rightarrow (0,0)$
1	$(1,0)$	$[(0,1), (2,0), (1,1)]$	5	$(2,0) \rightarrow (1,0); (1,1) \rightarrow (1,0)$
2	$(0,1)$	$[(2,0), (1,1), (0,2)]$	6	$(0,2) \rightarrow (0,1)$
3	$(2,0)$	$[(1,1), (0,2), (3,0), (2,1)]$	8	$(3,0) \rightarrow (2,0); (2,1) \rightarrow (2,0)$
4	$(1,1)$	$[(0,2), (3,0), (2,1), (1,2)]$	9	$(1,2) \rightarrow (1,1)$
5	$(0,2)$	$[(3,0), (2,1), (1,2)]$	9	Ninguno: vecinos ya registrados o bloqueados
6	$(3,0)$	$[(2,1), (1,2), (4,0), (3,1)]$	11	$(4,0) \rightarrow (3,0); (3,1) \rightarrow (3,0)$
7	$(2,1)$	$[(1,2), (4,0), (3,1), (2,2)]$	12	$(2,2) \rightarrow (2,1)$

El paso 5 muestra el papel de `parent` como filtro de duplicados. Al expandir $(0,2)$, su vecino de arriba queda fuera del mapa, su vecino de la derecha, $(0,3)$, es un obstáculo, y sus vecinos $(1,2)$ y $(0,1)$ ya estaban registrados (por $(1,1)$ y por $(0,1)$, respectivamente), por lo que no se agrega ningún estado nuevo. Así se evita procesar el mismo estado más de una vez, sin importar cuántos caminos distintos lleguen a él.

La ejecución completa expande 43 estados y termina en el paso 42, cuando se retira $(6,6)$ de la frontera y la prueba de meta resulta verdadera. Al reconstruir el camino desde los padres se obtiene la secuencia $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (4,4) \rightarrow (5,4) \rightarrow (6,4) \rightarrow (6,5) \rightarrow (6,6)$, es decir, 12 movimientos. Este resultado coincide con la distancia Manhattan entre $(0,0)$ y

(6,6), $|6 - 0| + |6 - 0| = 12$, calculada a mano, lo que verifica de forma independiente que BFS encontró una ruta óptima. Como el único hueco del muro está en la fila 3, el camino necesariamente pasa por (3,3).

Plan de pruebas y análisis de resultados

Casos de prueba

Para comprobar el funcionamiento del algoritmo y de la aplicación se utilizaron tres mapas con características distintas:

Caso principal. Mapa de 25×25 celdas generado con la semilla de las matrículas del equipo, con 28 % de obstáculos y una ruta de solución de 52 movimientos. Permitió comprobar el funcionamiento general: generación del mapa, ejecución de BFS, registro de visitados, visualización de la frontera y reconstrucción del camino.

Caso sencillo. Mapa de 7×7 celdas diseñado para la verificación manual. La ruta obtenida tiene 12 movimientos y, por su tamaño reducido, fue posible seguir el recorrido paso a paso (sección 6) y compararlo con el cálculo manual.

Caso de estrés. Mapa de 30×30 celdas con 35 % de obstáculos, el máximo permitido. Permitió observar el comportamiento del algoritmo en un mapa más grande y con menos espacio libre. La ruta encontrada tiene 60 movimientos.

Tabla de resultados

La Tabla 5 resume las métricas registradas en `04_resultados/metricas.csv`, generado con `ejecutar_pruebas.py`.

Tabla 5- *Resultados de BFS en los tres mapas*

Mapa	Tamaño	Obstáculos	Longitud de solución	Nodos generados	Nodos expandidos	Estados visitados	Frontera máxima	Tiempo (ms)
Principal	25×25	28 %	52	440	439	440	26	3.167
Sencillo	7×7	12.2 %	12	43	43	43	7	0.129
Estrés	30×30	35 %	60	391	390	391	17	1.999

BFS encontró solución en los tres mapas. Los conteos de nodos, la frontera máxima y la longitud del camino son deterministas: se repiten en cada ejecución. El tiempo, en cambio, depende de la ejecución

y del equipo; por eso los tiempos de la Tabla 6 son los de una corrida de `ejecutar_pruebas.py` y no deben leerse como valores exactos.

Evidencia visual

Las siguientes figuras muestran la interfaz al terminar la búsqueda en cada mapa. Los tiempos que aparecen en el panel de métricas de la interfaz (3.167 ms, 0.129 ms y 1.999 ms, respectivamente) son mayores que los de `metricas.csv` porque la interfaz ejecuta BFS registrando cada paso (`registrar_pasos=True`), lo que agrega trabajo adicional al copiar la frontera y los visitados en cada paso.

Figura 2 - Mapa principal (25×25 , 28 % de obstáculos): búsqueda finalizada, camino de 52 movimientos

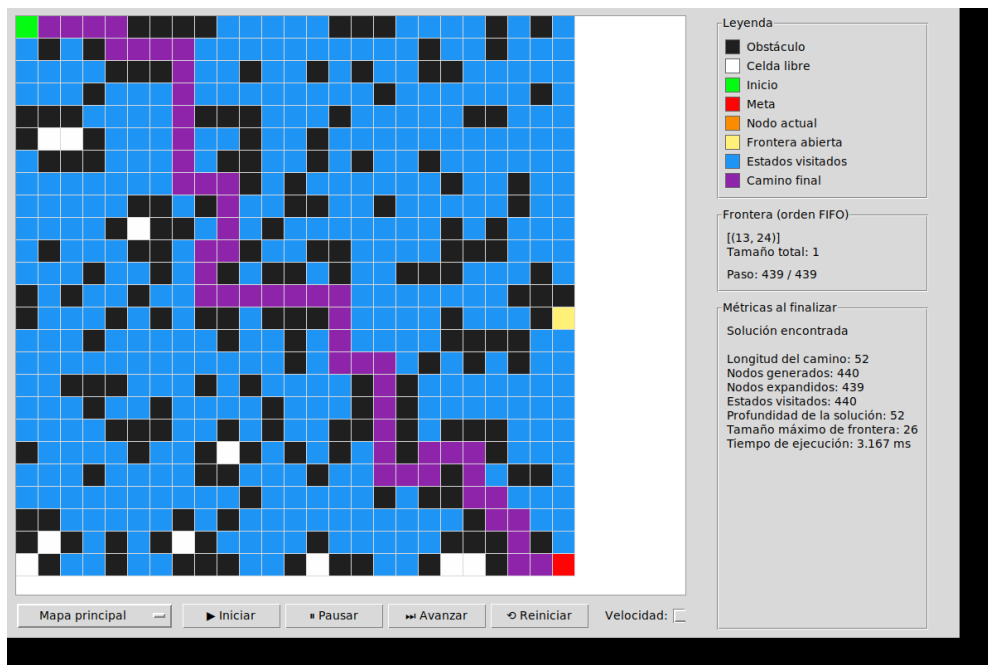


Figura 3 - Mapa sencillo (7×7): búsqueda finalizada, camino de 12 movimientos

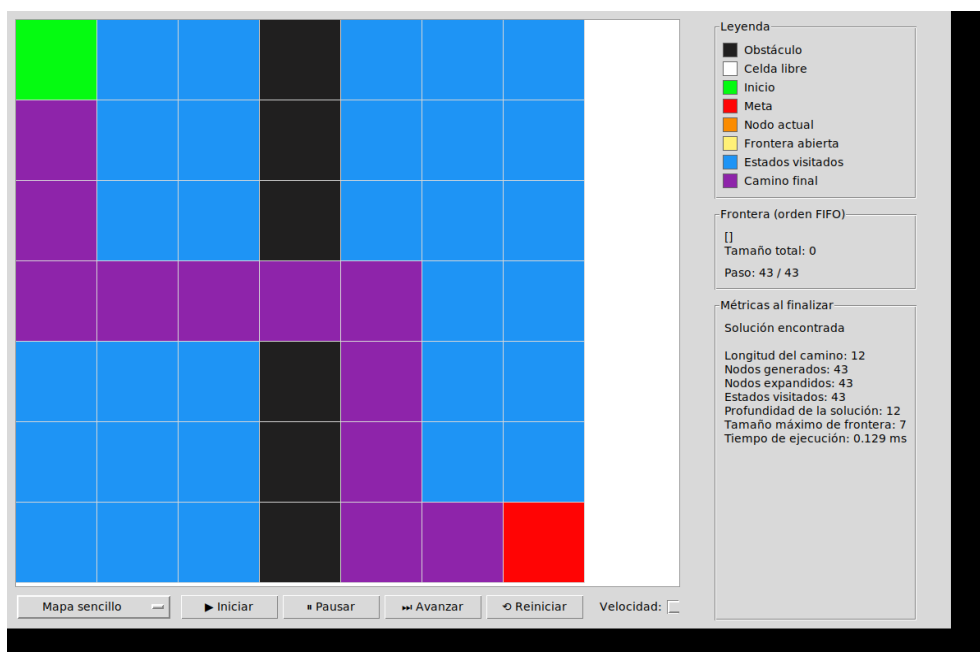
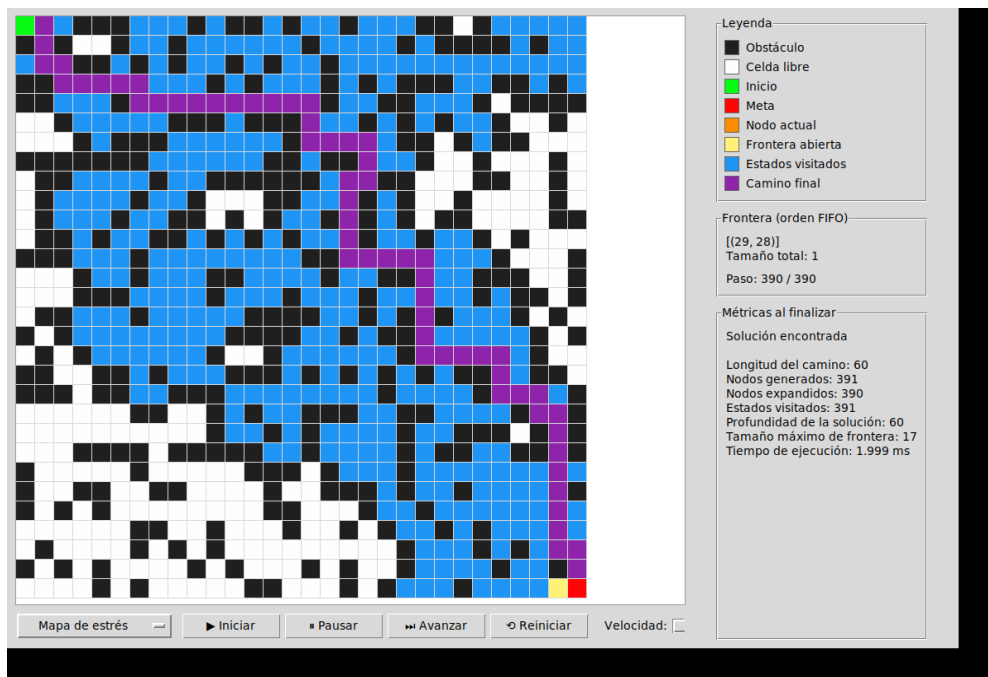


Figura 4 - Mapa de estrés (30×30 , 35 % de obstáculos): búsqueda finalizada, camino de 60 movimientos



Verificación de los resultados

Para comprobar que el camino obtenido fuera correcto, se revisó que cada movimiento conectara dos celdas transitables, que no hubiera movimientos diagonales y que el recorrido comenzara en la posición inicial y terminara en la meta.

También se contrastó la consistencia de las métricas. Al terminar la búsqueda, los nodos generados son iguales a los expandidos más los que quedan en la frontera: $440 = 439 + 1$ en el mapa principal, $43 = 43 + 0$ en el sencillo y $391 = 390 + 1$ en el de estrés, lo que coincide con el tamaño de la frontera final que muestra la interfaz (1, 0 y 1, respectivamente).

En el mapa sencillo, la ruta obtenida fue de 12 movimientos, igual a la distancia mínima calculada manualmente entre (0,0) y (6,6). Esta comparación también está implementada en `ejecutar_pruebas.py`, que emite un error si la longitud calculada no coincide con 12.

Análisis de métricas

En el mapa principal se generaron 440 nodos, de los cuales 439 fueron expandidos, con 440 estados visitados. La frontera alcanzó un máximo de 26 estados y la solución tuvo una profundidad de 52

movimientos. En el mapa sencillo se generaron y expandieron 43 nodos, con 43 estados visitados, una frontera máxima de 7 estados y una profundidad de 12 movimientos. En el mapa de estrés se generaron 391 nodos, se expandieron 390, se visitaron 391 estados, la frontera máxima fue de 17 estados y la profundidad fue de 60 movimientos.

Al comparar los tres casos, el mapa principal presentó la mayor cantidad de nodos generados y la frontera máxima más grande, mientras que el sencillo presentó los valores más bajos. El mapa de estrés, aunque es el de mayor tamaño y tiene la ruta más larga, generó menos nodos y tuvo una frontera menor que el principal; una explicación probable es que su mayor porcentaje de obstáculos deja menos celdas libres por las que la frontera pueda crecer. Esta explicación es una interpretación de los datos y no se probó con experimentos adicionales.

Respecto al tiempo, en la corrida de `metricas.csv` el mapa principal fue el más lento (1.113 ms), seguido del de estrés (0.313 ms) y del sencillo (0.056 ms), en concordancia con la cantidad de estados procesados. Se trata de una sola ejecución por mapa, por lo que estas diferencias son indicativas.

Explicación de completitud y optimalidad

BFS encontró una solución en los tres mapas. Mientras exista un camino transitable entre el inicio y la meta, el algoritmo sigue explorando hasta encontrar la meta o agotar los estados alcanzables (completitud). Si la frontera se vacía sin encontrar la meta, el programa lo reporta como “Sin solución”.

La optimalidad puede observarse en el mapa sencillo. La distancia mínima entre (0,0) y (6,6), considerando solo movimientos arriba, abajo, izquierda y derecha, es $|6 - 0| + |6 - 0| = 12$ movimientos, y la ruta encontrada por BFS tiene exactamente 12.

En los tres mapas cada movimiento tiene el mismo costo, y BFS explora por niveles y prueba la meta al retirar cada estado de la cola. Por eso encuentra primero las soluciones con menor número de movimientos, y la ruta reconstruida con los padres es de longitud mínima (Russell &

Norvig, 2022). Esta propiedad se verificó manualmente solo en el mapa sencillo; en el principal y en el de estrés se apoya en la propiedad teórica del algoritmo.

Limitaciones, mejoras posibles y conclusiones

Limitaciones identificadas

- Semilla fija en el código. Las matrículas están escritas directamente en `main.py` y en `generar_mapas.py`; para cambiar de semilla hay que editar el código fuente.
- Consumo de memoria del registro de pasos. Para animar la búsqueda, la interfaz guarda en cada paso una copia de la lista de visitados y de la frontera. En mapas mucho más grandes, este registro crece rápidamente y también aumenta el tiempo medido en la interfaz.
- Mapas externos sin validación. La opción “Cargar archivo...” lee el JSON sin comprobar su estructura ni que cumpla las condiciones del proyecto (tamaño, porcentaje de obstáculos o conectividad). Solo si el inicio o la meta son obstáculos el algoritmo lo detecta y reporta que no hay solución.
- Alcance de la evaluación. Solo se usó BFS, con movimientos ortogonales de costo uniforme y sin heurística, y cada mapa se midió con una sola ejecución. La optimalidad se verificó manualmente solo en el mapa sencillo.

Mejoras posibles

- **Aumentar el tamaño de los mapas:** permitir mapas de mayores dimensiones para observar el comportamiento de BFS en escenarios más amplios.
- **Modificar la distribución de obstáculos:** permitir ajustar la cantidad de obstáculos desde la interfaz para generar distintos escenarios de prueba.
- **Parametrizar la semilla:** capturar las matrículas como argumento o desde un archivo de configuración, sin modificar el código.

-
- **Guardar los resultados de cada ejecución:** actualmente solo `ejecutar_pruebas.py` genera `metricas.csv` (y lo sobrescribe); se podría registrar cada ejecución de la interfaz con su fecha y el mapa utilizado.
 - **Comparar con otros algoritmos:** como trabajo futuro, comparar BFS con otros algoritmos de búsqueda en cuanto a recorrido, estados explorados y tiempo de ejecución.

8.3 Conclusiones individuales

Iván Bautista Martínez

Durante el desarrollo de este proyecto se buscó comprender y aplicar el algoritmo BFS mediante la exploración de mapas. Su implementación permitió conocer cómo se utilizan las colas, los estados visitados y el registro de padres para encontrar y reconstruir una ruta desde el inicio hasta la meta.

Además, se realizaron pruebas con diferentes mapas para comprobar el funcionamiento del algoritmo y analizar los resultados obtenidos. Esto permitió relacionar los conceptos vistos en clase con una aplicación funcional y fortalecer los conocimientos sobre algoritmos de búsqueda y programación en Python.

Isaí Pascual Cruz

En el desarrollo de este proyecto se tardo de aplicar y entender el BFS, el cual es un algoritmo utilizado para recorrer o buscar los elementos de un grafo. Esto viene siendo muy útil para encontrar el camino más corto y eficiente.

Contribuciones, declaración de uso de IA y referencias

Tabla de contribuciones

Tabla 6 - Contribuciones de los integrantes

Integrante	Actividades realizadas	Porcentaje de participación	Firma
Isaí Pascual Cruz	Desarrollo conjunto de la aplicación, participando en la creación, programación e integración del código, así como en la realización de pruebas y validación de su funcionamiento. Elaboración conjunta, organización y revisión de la documentación del proyecto.	50 %	
Iván Bautista Martínez	Desarrollo conjunto de la aplicación, participando en la creación, programación e integración del código, así como en la realización de pruebas y validación de su funcionamiento. Elaboración conjunta, organización y revisión de la documentación del proyecto.	50 %	

Declaración de uso de herramientas de IA

Durante la elaboración de este proyecto se utilizaron herramientas de Inteligencia Artificial como apoyo en diferentes etapas del trabajo. ChatGPT se utilizó principalmente como apoyo para la redacción, organización y revisión de algunas partes de la documentación del proyecto.

En la etapa de programación, Claude se utilizó como herramienta de apoyo para resolver algunas partes del código que presentaron dificultades durante el desarrollo. Las sugerencias obtenidas fueron revisadas, adaptadas y probadas por los integrantes para comprobar su funcionamiento dentro de la aplicación. Claude también se utilizó para completar y revisar la redacción de este reporte a partir de los archivos del proyecto; el contenido fue revisado por los integrantes.

El uso de estas herramientas fue únicamente como apoyo durante el desarrollo. La programación, integración, realización de pruebas, análisis de resultados y elaboración final del proyecto fueron realizadas y revisadas por los integrantes del equipo.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Juárez Castillo, E. (2026a). *Búsqueda en amplitud* [Diapositivas de clase, Sesión 7]. Fundamentos de Inteligencia Artificial, Universidad Tecnológica de la Huasteca Hidalguense.
- Juárez Castillo, E. (2026b). *Problemas y espacios de búsqueda* [Diapositivas de clase, Sesión 6]. Fundamentos de Inteligencia Artificial, Universidad Tecnológica de la Huasteca Hidalguense.
- Python Software Foundation. (s. f.-a). *collections - Container datatypes*. Python 3 documentation. <https://docs.python.org/3/library/collections.html>
- Python Software Foundation. (s. f.-b). *json — JSON encoder and decoder*. Python 3 documentation. <https://docs.python.org/3/library/json.html>
- Python Software Foundation. (s. f.-c). *random — Generate pseudo-random numbers*. Python 3 documentation. <https://docs.python.org/3/library/random.html>
- Python Software Foundation. (s. f.-d). *time — Time access and conversions*. Python 3 documentation. <https://docs.python.org/3/library/time.html>
- Python Software Foundation. (s. f.-e). *tkinter — Python interface to Tcl/Tk*. Python 3 documentation. <https://docs.python.org/3/library/tkinter.html>
- Russell, S., & Norvig, P. (2022). *Artificial intelligence: A modern approach* (4.^a ed., edición global). Pearson Education.